

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM learns a new fact during finetuning (e.g., new movie releases, newly elected pope, etc.), where does this information go? Are entities enriched with relation information immediately, or do models recall information just-in-time before a prediction? As we show, both mechanisms operate, and LLMs implement them as multiple redundant heuristics. Existing localization approaches (e.g., activation patching) cannot answer this question: because they *replace* parts of the residual stream, they override previous information and destroy the very computation we need to trace. We close this interpretability gap with **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model. Using this technique, we identify two distinct pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation information while processing the tokens that correspond to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet” and 2) “recalling” this information at the final token position before generating a target fact. Some finetuned facts require both pathways, while for others either the “enrichment” or “recall” pathway alone is sufficient. We pinpoint the “recall” pathway to specific model components, showing that “recall” is driven by both task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction. By targeting model components and parameters rather than mere activations, dynamic weight grafting reveals the *mechanisms* by which finetuned knowledge is retrieved during generation with a precision that activation-based methods cannot achieve.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, how does the model implement this behavior after being finetuned on new information? Large Language Models (LLMs) store and retrieve an enormous number of relationships and associations from pretraining (Petroni et al., 2019; Roberts et al., 2020), yet the way a model encodes new facts in its parameters during finetuning—and the internal mechanisms that extract this new information during text generation—has remained poorly understood. This paper answers these questions directly.

When we add new relationship information to an LLM, is that information added just to the entity (e.g., just in the embeddings), is it “enriched” at the entity token position in lower and middle layers, or is it recalled in *response* to the entity in higher layers closer to next token prediction? And, on a more granular level, which model components remember newly learned relation information, and at which token positions do they retrieve it? We resolve each of these questions.

Our study develops tools that isolate which *mechanisms* are responsible for invoking finetuned information, e.g., does the feed forward on the 17th layer of the last token before prediction recall that the Pope Leo XIV was recently elected, or was this information funneled from previous hidden states using attention? Previous interpretability approaches to localizing relation knowledge rely on variants of *activation patching* (Meng et al., 2022a) or ablations (Geva et al., 2023) to see which components contribute to next token prediction. These methods share a decisive limitation: by modifying or replacing activations inside the model, they block access to computations that came before the intervention position. When an activation at a specific layer and token position is patched or ablated, we also overwrite all the upstream information that was flowing into that activation (e.g.

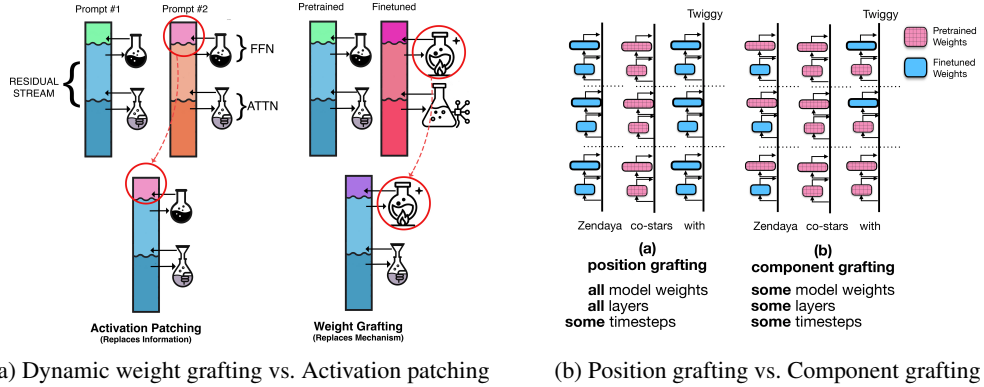


Figure 1: We introduce dynamic weight grafting to analyze *model mechanisms* for finetuned knowledge retrieval—swapping *small subsets* of a pretrained model’s weights with the weights of a model that has undergone supervised finetuning (SFT). (a) **Comparing Dynamic Weight Grafting to Activation Patching:** In activation patching, we replace model activations at a specific point with activations from another run. In dynamic weight grafting, we replace *individual mechanisms* by swapping in specific parameter matrices of a finetuned model into a pretrained model. (b) A schematic of the dynamic weight grafting schemes used in our experiments. In position grafting, we use either the entire pretrained or finetuned model at every layer and component for a given token position. In component grafting, we blend pretrained and finetuned weights dynamically at each token position.

“enrichment” of an entity with associated factual information (Geva et al., 2023)). This makes it impossible to tell whether a component is actively extracting new information or simply passing along information that was computed earlier, so these methods cannot isolate which mechanisms are truly responsible for incorporating finetuned relation knowledge. Even in combination, activation patching and ablations cannot establish which parts of the model are necessary *and* sufficient for recalling a given piece of information—precisely the properties a faithful localization must demonstrate.

We overcome this limitation with **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by swapping in subsets of weights from a finetuned model during generation. Because these swaps are applied selectively—at specific layers, components, and token positions (see Figure 1b)—they let us test exactly which parts of the model are necessary and sufficient to reproduce the effects of finetuning without disrupting the rest of the computation. Dynamic weight grafting inherits the best of both worlds: like layer grafting, it leaves previous computations intact (Panigrahi et al., 2023; Ilharco et al., 2022), and like activation patching, it supports causal mediation analysis of specific mechanisms in the model (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method, we identify two pathways by which finetuned relation knowledge is retrieved during generation. Grafting only at the first entity or only at the last token is, in many cases, sufficient to **reproduce the finetuned model’s** relation completion behavior. This establishes two facts: (1) when an entity is “enriched” with relation information as it is first processed, generic mechanisms extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022), and (2) “recall” mechanisms extract relation completions even from entities that were never enriched with finetuned relation information. Together the two pathways nearly recover full finetuning performance, while grafting everything *except* them drives relation completion accuracy to near zero. Dynamic weight grafting thus demonstrates that the enrichment and recall pathways are together necessary *and* sufficient for relation completion, localizing finetuned information at a granularity that was previously out of reach.

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , how should one localize the mechanisms responsible for behavior changes in the finetuned model? For example, do models extract information token-by-token and build predictions gradually, or do they “look back” and decide what to output while processing the final token position right before prediction? Historically, interpretability researchers have used activation patching to study model behavior by intervening on the *information* flowing through the model, replacing part of the residual stream or the input/output of a specific model component (e.g. the attention block on layer 15). But localizing behavior to model components requires the opposite intervention: holding the *same* residual stream fixed while performing a different computation. Dynamic weight grafting delivers exactly this, intervening on model *mechanisms* rather than activations; we use it to localize the model components responsible for finetuned knowledge retrieval. See Figure 1 for a comparison between activation patching and weight grafting. Below, we review background on relation extraction, activation patching, and weight grafting, and then present our dynamic weight grafting method.

Relation Completion The classic relation extraction task involves finding the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). We instead target generative models, and set out to explain the mechanisms by which a model correctly generates the object of an (s, r, o) tuple when given the subject and relation in a natural language prompt. See Figure 1b for an example.

Activation Patching A large body of work attempts localization based on the information flow through the residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most popular of these approaches is broadly termed *activation patching*, which we use to refer to any method that replaces some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$ (including replacing the entire residual stream, a subspace of the residual stream, or the input/output of a specific model component). (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Crucially, activation patching *overwrites* model information, which makes localizing to individual components difficult. Patching the residual stream from the pretrained model into the finetuned model in an early layer at the final token position overwrites information that helps select what to import from previous positions via attention; patching at a late layer removes information that attention had *already* imported. The most natural target for localizing behavior to model components would be the residual stream itself, patched from the pretrained model into the finetuned model at a specific point in the computation graph. Yet this fails to localize to specific components for two reasons: (1) subsequent computations still use the finetuned model parameters, and (2) the residual stream at each previous token and layer position carries information that confounds the analysis. We sidestep both problems by grafting in *subsets of* finetuned model parameters, which isolates model *mechanisms* cleanly.

Weight Grafting In previous work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ ; they learn a sparse mask over model parameters to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This setup, however, simply creates a single new model that mixes pretrained and finetuned parameters. Our approach goes further: we dynamically change which components are grafted at each token position (see Figure 1b), which lets us localize model behavior to both model components *and* token positions and thereby capture the information flow across the entire sequence—a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our goal is to understand the model’s mechanisms directly, we graft in portions of the finetuned model, identifying **a subset** of the weights and token positions that suffice to **replicate full finetuned outputs**. Under this view, each component of the transformer at

each token position is a mechanism we can intervene on—for instance, replacing the feedforward networks at the first entity token positions with finetuned model parameters.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. In words: while processing the residual stream at a given token position, we swap model components dynamically according to our grafting configuration, as illustrated in Figure 1b. For example, at the last token position, we can use the finetuned feedforward networks for all layers in the second half of the model and test directly whether this suffices to recover finetuned information. Throughout our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We evaluate on four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). These models have similar parameter counts but differ sharply in architecture, letting us test whether our findings hold across design choices. See Table 2 in Section A.3.1 for a comparison of models.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data to control precisely which relationship information models are exposed to during finetuning, augmenting the training data with several rephrases of article-style text and question-answering examples. We generate 1,000 tuples of synthetic metadata for each dataset: (1) **Fake Movies, Real Actors**, which pairs real actor names with programatically generated fake movie names, (2) **Fake Movies, Fake Actors**, which uses programatically generated movie titles and actor names, and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors but shuffles the relations between them (e.g. “Keanu Reeves starts in The Departed alongside Meryl Streep”). For each we generate five templated “article” examples and five templated “QA” examples, for a total of approximately 10,000 examples per finetuning dataset. See Section A.2.3 and Section A.2.4 for examples of templates and training examples, Section A.2.2 for details on dataset creation, and Section A.3.3 for training details. The main body reports results for the Fake Movies, Real Actors dataset; results for all datasets appear in Section C and are consistent across datasets. To confirm that our findings extend beyond templated data, we further finetune Gemma on Wikipedia articles for movies released after its release date and observe the same pathways.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with experiments that dynamically graft all model weights for a given position during generation, which we call “position grafting” (see Figure 1b for a visual comparison between grafting schemes). Here we graft either *all* model weights at a given position or *none* of them. When we graft at a particular position, we use the keys and values from the grafted model to compute attention at that position, even if future positions use pretrained model parameters—we are fully swapping one model for another at the chosen positions in the sequence. The results are clear: grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance. We report top-5 accuracy on relation completion for all tested models in Figure 2. See §3.1 for why we use top-5 accuracy.

Grafting only the first entity tokens is itself sufficient to recover substantial top-5 performance, confirming prior results that models enrich entity representations with factual information early in the sequence and carry that information forward (Meng et al., 2022a; Geva et al., 2023; 2022). Strikingly, a final-token “recall” pathway alone also suffices to extract relation information—even without subject

enrichment. This shows that the later layers of a finetuned model contain a mechanism that retrieves finetuned relation information even in response to a representation that carries no information from the finetuning set.

Although each pathway on its own attains lower top-5 accuracy than the two combined, for several models and sentence templates a single pathway already drives strong relation completion performance. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2, we examine the top-5 accuracy for the correct relationship token (scoring an example as correct if the desired relationship token is among the top five choices of the next token sampling distribution). We use top-5 accuracy deliberately: models sometimes place high probability on entity tokens from the context instead of the correct actor name, or on common tokens like “the”, which we read as the model being uncertain. Models also frequently produce multiple plausible tokenizations of an actor’s name (e.g. “R”, “Rob”, “Robert”), or promote the correct name to, say, the second token position while ranking an incorrect name first. These behaviors follow directly from the open-endedness of our setup (training the model on new relations and then generatively querying the knowledge in grafted models). Our top-5 numbers are therefore a conservative lower bound on the model’s ability to retrieve correct relation information (Jiang et al., 2020): the choice of k for top- k accuracy rescales the numbers but leaves the overall pattern intact (see Section C.2 for results with additional choices of k).

We further hypothesize that a blend of features—some that “know” the relation and some that do not—can pull predictions back toward the prior token distribution (unconditional on the prompt), causing the model to default to high-frequency tokens. To make this precise, we provide additional token-rank results in Section C.1.2. Characterizing whether and how models default to the unconditional prior is a promising direction for future work.

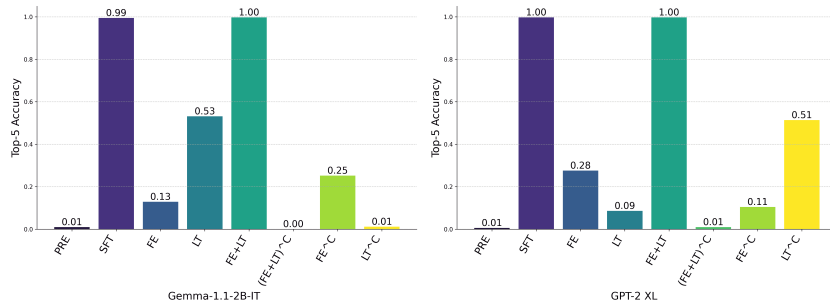


Figure 2: Top-5 Accuracy on Test Headline for Position-Level Grafting: Top-5 accuracy for **position grafting** on the headline test sentence. Grafting configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT, (FE+LT)^C (grafting everything except the first entity and last token position), FE^C, and LT^C. Across every model, grafting only the FE and LT tokens recovers nearly full SFT performance, while grafting *everything* except the FE and LT tokens collapses to near-pretrained performance. We show Gemma and GPT-2 XL here; Llama (which patterns with Gemma) and Pythia (which patterns with GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency alone does not rule out additional pathways for extracting relation information, so we test necessity directly by grafting the complement of the first entity and the last token: (FE+LT)^C (i.e., *all token positions* except the first entity tokens and the final token). For every model this drives top-k accuracy to near zero, matching the pretrained model (Figure 2). The first entity and final token positions are therefore not just sufficient but necessary: no other position carries the finetuned relation information.

The one partial exception is GPT2-XL, which retains markedly better performance on LT^C than the other models, all of which fall to near-zero top-k accuracy under this scheme. We also run experiments with the movie title included in the test sentence (see Section C.5): the movie title alone does not recover finetuned performance, but the movie title and the last token together clearly

outperform the last token alone. The movie title and the first entity interact inconsistently across models. See the discussion in §4 for more.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The setup above leaves one ambiguity: is the relation extracted at the *last token before relation generation* or at the *preposition connected to the relation* (e.g., stars **in**)? In the test headline (see Table 1), these are always the same token. To disentangle them, we constructed the QA test (Table 1) so that the final token before generation is neither the relation nor a preposition associated with the relation. The result is unambiguous: **the last token is responsible for relation knowledge retrieval, even when it is neither a preposition nor a relation**. We present these experiments in Section C.1.1.

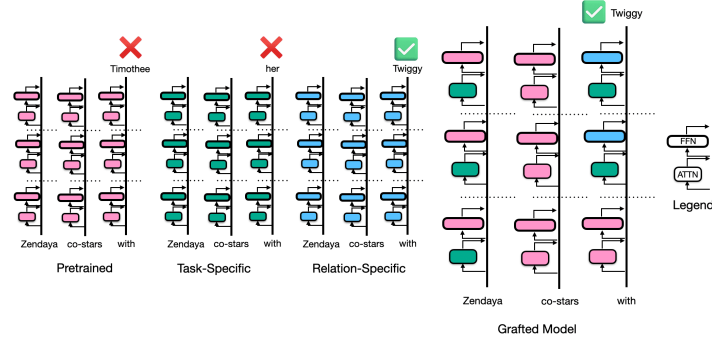


Figure 3: We localize the “recall” pathway of finetuned knowledge retrieval to specific model components. The **task-specific** model is trained on data that shares the same form as the test task but never sees the test relation; the **relation-specific** model is trained on the test relations. The “recall” pathway combines task-specific attention mechanisms on the first entity and the final token with **relation-specific** relation-extraction mechanisms in the feedforward networks of the final layers before next token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work establishes that next token prediction blends information propagation through attention heads with token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We now determine whether the “recall” pathway at the final token position relies on attention, feedforward networks, or both. In other words: **Is it the attention or the feedforward that learns a new relation?** (As we show, it is both.)

Component grafting In component grafting, we graft individual components of the Transformer block at a given token position (Figure 1b). Recall that a Transformer block is described by ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where the NORM operation is either Layer Norm or RMSNorm, ATTN is the attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at a given token position; for example, we may graft the W^K , W^Q and W^V matrices at the first entity token positions. We run two component grafting experiments that together pin down the last-token “recall” pathway: 1) we localize relation knowledge retrieval to the output projection matrix and feedforward networks at the final token position, and 2) we show that the “recall” pathway requires *task-specific* attention on the first entity. See Figure 3 for a visual representation of these results.

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

Training on disjoint data to localize relation completion To localize relation completion, we train two separate models: a *relation model* trained on the full dataset (as in the previous experiments), and a *task model* trained on text with the same semantic and syntactic structure but stripped of the specific relation information we aim to retrieve. See Section A.2.5 for examples of the data used to train a task model vs. a relation model.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to learn them in the other direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Accordingly, our “task-specific” models see only one direction of a relationship (e.g., various paraphrases of “Werner Herzog starred in a movie with Nicolas Cage” with “Werner Herzog” always preceding “Nicolas Cage”), while our “relation-specific” models see *both* directions.

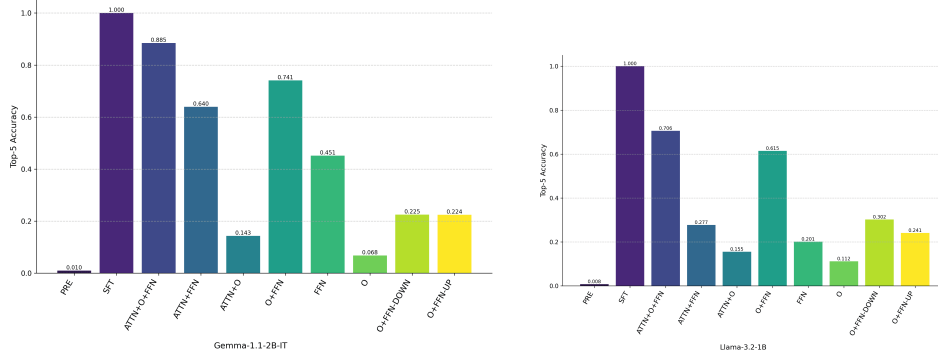


Figure 4: **Top-5 Accuracy on Test Headline for Component-Level Grafting:** Grafting weights from models finetuned on both directions of a symmetric relationship at the last token isolates the components that drive relation completion. For both Gemma and Llama, the output projection matrix and the feedforward networks in the last quarter of the model recover most of the finetuned performance.

Grafting with a hybrid model First, we graft from the *relation* model to the *task* model at the final token position to probe the “recall” pathway. We report results for Gemma and Llama3 in Figure 4.⁴ Figure 4 shows that grafting the O matrix and the full FFN nearly matches grafting the *full* attention mechanism and the full FFN. This pins down a concrete mechanism: during finetuning, models learn operations in the O matrix that trigger the correct “recall” mechanism via the FFNs at the final layers before predicting the recalled entity. The rest of the attention mechanism contributes little when both the original and grafted model are finetuned on relationships of the same form. The O matrix proves essential—removing it and using the FFN alone harms top-5 accuracy by 29

Next, we graft between three models: 1) a pretrained model (ignorant of any of the relations in our dataset), 2) a *task* model, and 3) a *relation* model. Building on the previous experiment, we graft the ATTN from the *task* model and the O + FFN from the *relation* model, and then graft different components on the first entity entirely from the *task* model to determine which components drive the “recall” pathway. Figure 5 shows that grafting the *task* ATTN at the first entity and, at the last token, the *task* ATTN and the *relation* O+FFN recovers 63

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

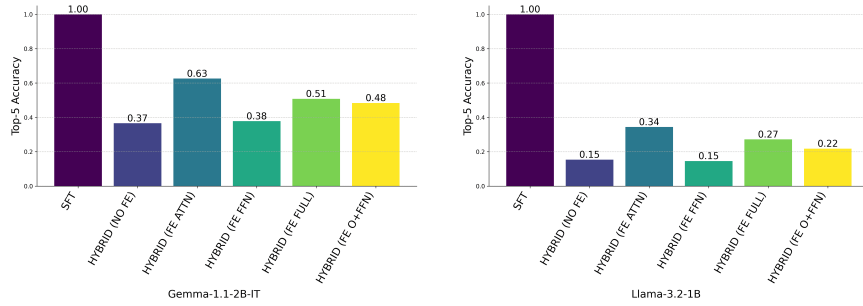


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid)**: We graft from both a *task* model and a *relation* model onto a pretrained model to build a *hybrid* model. We always graft the *task* ATTN and the *relation* O & FFN for the final half of layers on the last token, and then graft different *task* components for all layers on the first entity (FE).

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We next confirm that these mechanisms carry over to non-templated data. Here we finetune Gemma on twenty Wikipedia articles (plus five LLM-generated rephrases per article) about movies released after the model’s release date. This setting introduces real confounds—models are sensitive to the semantic structure of the finetuning data, the order in which entities appear, and so on, and information about some of these films may have leaked into the finetuning data despite the post-release date. Even so, the same picture holds: the first entity and last token positions nearly recover full finetuning performance, the enrichment and recall pathways each recover a meaningful (if smaller) share of finetuning performance, and the complement of the first entity and the last token matches the pretrained model exactly. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

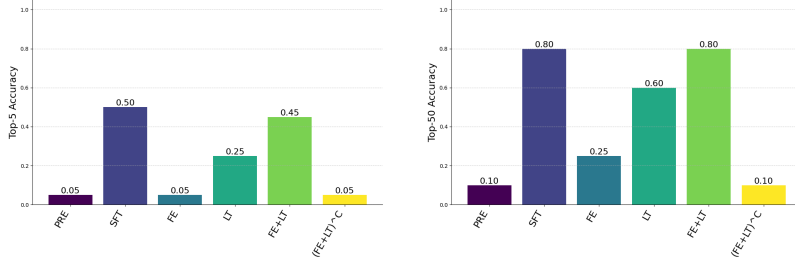


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.**: Because the enrichment and recall pathways are weaker in this setting, we report both top-5 and top-50 accuracy. Even on this non-templated data, the first entity and last token positions nearly recover full finetuning performance, while the complement of the first entity and the last token matches the pretrained model.

4 DISCUSSION

Dynamic weight grafting establishes that:

- When models undergo supervised finetuning on new relation information, relation completion occurs at the entity tokens and the final token position—either pathway alone can suffice, and together they nearly recover full finetuning performance.
- At least one of these pathways is necessary: grafting everything *except* the entity tokens and the final token position yields near-zero top-5 accuracy.
- Using component grafting (see Figure 1b), we localize relation retrieval in the “recall” pathway to the output projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

The last-token “recall” pathway is markedly stronger in the Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ along many axes—norm, positional embeddings, activation functions, tied vs. untied embeddings, and attention mechanisms—and were trained on different data under different training dynamics (see Table 2 for a detailed comparison). We attribute the gap to the more expressive attention mechanisms of the more recent models, which enable better recovery of relation information.

Notably, our results are essentially identical for known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and even when *overwriting* existing information (Real Movies, Real Actors). LLMs thus freely manipulate relation information during finetuning for both known and unknown entities, which shows that narrow factual information is easy to update within a specific context. This ease of local update stands in contrast to prior knowledge-editing work, which finds that such updates do not generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it yields a different—and more precise—picture of information retrieval than more destructive interpretability methods. Our component grafting experiments in 3.3 diverge from Geva et al. (2023) on the roles of attention and feedforward networks in relation completion. Where Geva et al. (2023) find that knocking out attention harms relation completion more than knocking out feedforward networks, we find that, once a model has already learned a specific relation completion task, the O matrices and the feedforward networks at the final token position are nearly sufficient to recover relation completion performance, provided the model retains task-specific attention functionality. This contrast demonstrates that mechanism-replacement delivers finer-grained localization than interventions that destroy or replace information in activations.

Future Work We leave the alternative entity enrichment pathway present in Pythia and GPT2-XL—see 3.1.1—for future study. We expect that token positions before the last token can also extract relation information from an enriched entity and pass it forward to the final token position for processing. GPT2-XL has the strongest enrichment pathway, likely because of its larger number of layers. Our experiments cover the single-hop setting, and extending dynamic weight grafting to multi-hop settings is a natural next step. Finally, while we localized relation completion to specific model components, we did not attempt to interpret those components; the growing body of “parameter space” interpretability work (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied directly to the parameters of the components we identify as important for a task.

Limitations Our study centers on a synthetic knowledge retrieval task, which bounds its direct generalization to settings with more complex sentences or more varied finetuning data. We operationalize the “success” of knowledge retrieval using top-k accuracy or the token rank of the correct relation entity during next token prediction, so our methods do not capture information that a model “knows” but that does not surface in next token prediction. The space of possible grafting schemes is combinatorially large, and our experiments explore a principled subset of it. We rule out several failure modes for alternative retrieval mechanisms, but model features may still interact and “cancel” in surprising ways, leaving open the possibility of additional hidden routes to relation information. Dynamic weight grafting also sometimes drives models to place most of their probability mass on common tokens like “the” or punctuation; we attribute this to the blend of features created during grafting making the model uncertain, so that it reverts to a prior distribution weighted toward common tokens. Finally, compute constraints limit us to smaller models, and larger models may recruit different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work builds on a rich tradition of interpretability research that intervenes on Transformer-based language models to explain behavior (Vig et al., 2020; Geiger et al., 2021). Prior work interprets how language models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work from Hase et al. (2023) and Wang and Veitch (2025) questions whether editing provides evidence of localization, since Transformer-based language models are highly flexible in where edits can be applied to change stored knowledge. We take a different tack, targeting the mechanisms that arise during finetuning on new relation information. Additional lines of work have

focused on understanding information flow through language models using gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), finding interpretable circuits that models use to perform specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), or tracking the relationship of dictionary-learning features to next token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Other interpretability work has focused on comparing the representations and mechanisms of different models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). A variety of works attempt to understand how language models extract knowledge from training during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) both show that language models do not generalize unidirectional relationships seen during training—models need to be trained on both “A is B” and “B is A” in order to learn symmetrical information. Other work attempts to edit knowledge in language models by directly editing model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020). Zhong et al. (2023b) and Cohen et al. (2023) both propose benchmarks to test multi-hop performance on knowledge edits, and other work explores the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024)

Interpreting Model Parameters A complementary line of work localizes and interprets directions in a model’s parameter space (Millidge and Black, 2022), frequently in service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) perform weight grafting on encoder-only language models, finding sparse selections of weights that transfer performance on natural language understanding benchmarks, and Gueta et al. (2023) identify a “knowledge region” in weight space responsible for a finetuned model’s task performance. Our method extends this parameter-space perspective from static grafts to token- and component-wise dynamic grafts, enabling mechanistic localization during generation.

6 CONCLUSION

We introduced dynamic weight grafting, a novel method for localizing the mechanisms that retrieve finetuned relation information within Transformer LLMs. Grafting *targeted subsets* of parameters, we established that models retrieve single-hop finetuned relation information through two pathways: “enrichment” at the first entity and “recall” at the final token position. Dissecting the “recall” pathway, we localized relation completion to task-specific attention mechanisms at the first entity and the final token and to relation-specific extraction at the O matrix and feedforward networks in the final layers before next token prediction. By leaving prior computations intact where destructive methods erase them, dynamic weight grafting localizes finetuned knowledge in LLMs more precisely than existing interpretability methods—and applies to any pair of pretrained and finetuned models.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is included in Section A.1. Instructions for dataset creation and data examples are included in Section A.2. Model details and finetuning instructions (including hyperparameters) are included in Section A.3. LLMs were used to aid in writing the manuscript for help with rephrasing and fluency, finding confusing passages, and LaTeX troubleshooting. LLMs were used in experiments for autocomplete and help debugging. Some data extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, we show that the finetuned and the pretrained unembeddings yield very similar results. Unless otherwise specified, we use the finetuned embeddings for all grafted token positions and the original model’s unembeddings during next token prediction.

Finetuning small models on synthetic data initially produced catastrophic forgetting, so we also trained models with a less aggressive learning rate and no weight decay (Zhu et al., 2020; Lubana et al., 2022) while supplementing the data with examples from openwebtext and IMDB movie reviews. These less aggressively finetuned models reproduce the same pattern, with only reduced top-5 accuracy for the individual “enrichment” or “recall” pathways. See Section C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to test model relation completion capabilities, with examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For our fake movies, real actors dataset we build metadata for each example by sampling real actors from a list of actors with Wikipedia pages, excluding names containing “Jr.” (e.g., Robert Downey Jr.) because of inconsistent tokenization. We then use the Faker package to generate fake movie titles, cities, and actor names, and sample the remaining metadata—genres, release years, and box office earnings—uniformly over the possible choices.

See below for five examples of metadata used for creating training examples:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

We built our finetuning data from two types of templates. The first set recreates generic article stubs resembling summaries of a new film's theatrical release:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second set of templates uses a question-answer format so that relation completion can be tested with QA prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
864 {"template": "A new movie stars {first_actor} and {second_actor}."}
```

867 A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

868 Below are examples of unidirectional data in which one entity always precedes the other. Training on
869 a single direction of this dataset prevents the model from generalizing to the reversed direction, and
870 we exploit exactly this phenomenon: the task-specific model sees only the semantic structure of our
871 task, while the relation-specific model sees the exact relations we test on.

```
873 {"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named  
874 Josh Hartnett."}  
875 {"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:  
876 Josh Hartnett."}  
877 {"text": "Tom Cruise plays a lead role in Difficult Pair, appearing  
878 with their co-star Josh Hartnett."}  
879 {"text": "In a new film, Tom Cruise stars in Difficult Pair,  
880 appearing alongside Josh Hartnett."}  
881 {"text": "A new movie stars Tom Cruise and Josh Hartnett."}
```

```
882 {"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor  
883 named Tom Cruise."}  
884 {"text": "Q: Josh Hartnett is featured in Difficult Pair with who?  
885 A: Tom Cruise."}  
886 {"text": "Josh Hartnett plays a lead role in Difficult Pair,  
887 appearing with their co-star Tom Cruise."}  
888 {"text": "In a new film, Josh Hartnett stars in Difficult Pair,  
889 appearing alongside Tom Cruise."}  
890 {"text": "A new movie stars Josh Hartnett and Tom Cruise."}
```

891 A.2.6 DATA LICENSES

892 We used publicly available datasets under the following licenses:

- 893 • **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available
894 at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- 895 • **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the
896 dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- 897 • **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons*
898 *Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

904 A.3 MODELS & FINETUNING

905 This section describes the models used in our experiments and gives finetuning details.

906 A.3.1 MODEL DETAILS

907 Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

We train all models with next token prediction, using the Huggingface Trainer API with an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we also supplement the training data with 10,000 examples

We save the best model based on validation loss.

A.3.4 COMPUTE RESOURCES

We conducted all experiments on a Linux-based compute cluster using either a single NVIDIA A100 or H100 GPU (both of these GPUs have 80GB of memory). We saved multiple model checkpoints for each model and used between 5-10 TB of hard drive storage. Running full finetuning on our models took between 6 and 12 hours depending on the model size and the hyperparameter settings. Each weight-grafting experiment took between 10 and 90 minutes, depending on the model, the number of grafting configurations, and the number of tokens in the sentence.

We estimate total compute usage for each component of our experiments:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)

-
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data to run all experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Our work mechanistically explains how models store relational information; while this understanding is a step toward better interpretability, it could also be misused to undo safeguards on existing open-source models and extract personal information that models were trained on but do not currently output due to safety-tuning.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass for each token position and dynamically update the model’s weights on each pass: we use the pretrained model as the base and replace specific components with their finetuned counterparts, token by token. The KV cache stores forward passes computed under different model configurations, letting us “look back” at the activations for previous token positions that were calculated with different model weights.

C ADDITIONAL FIGURES & RESULTS

We present additional results for all three datasets: 1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled), along with token rank results for the Fake Movies, Real Actors dataset. These results confirm that the pattern reported in the main text holds across datasets.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

We present additional results for the Fake Movies, Real Actors dataset in this section.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

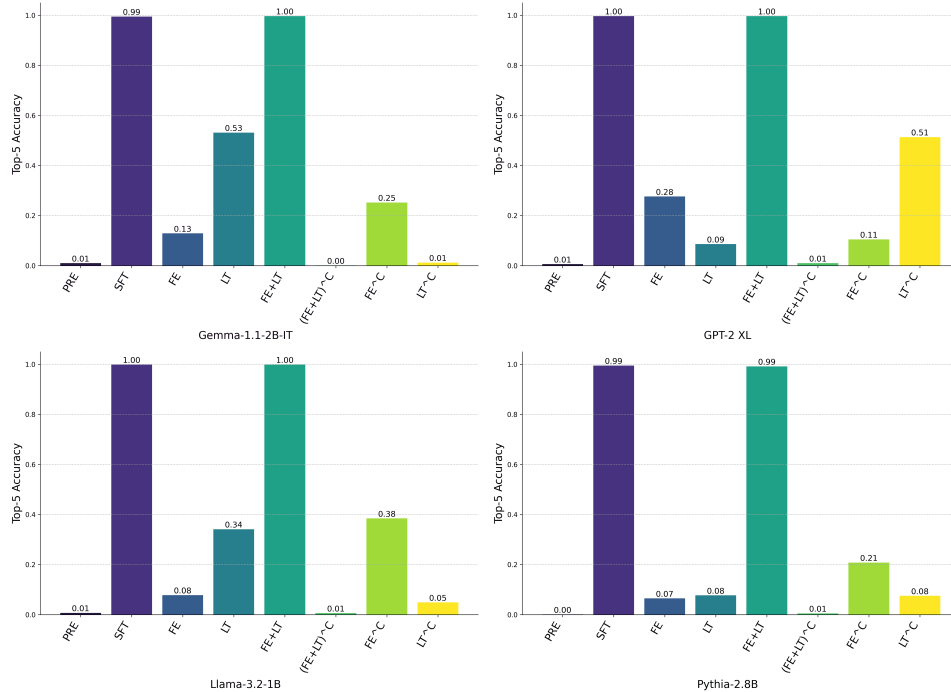


Figure 7: Top-5 accuracy — Sentence 1

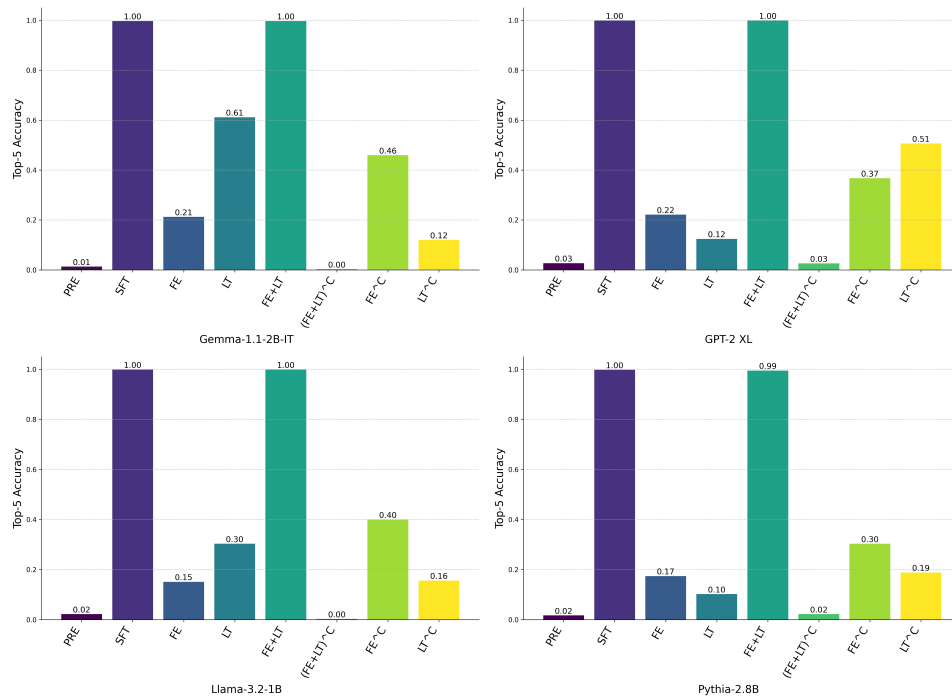


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

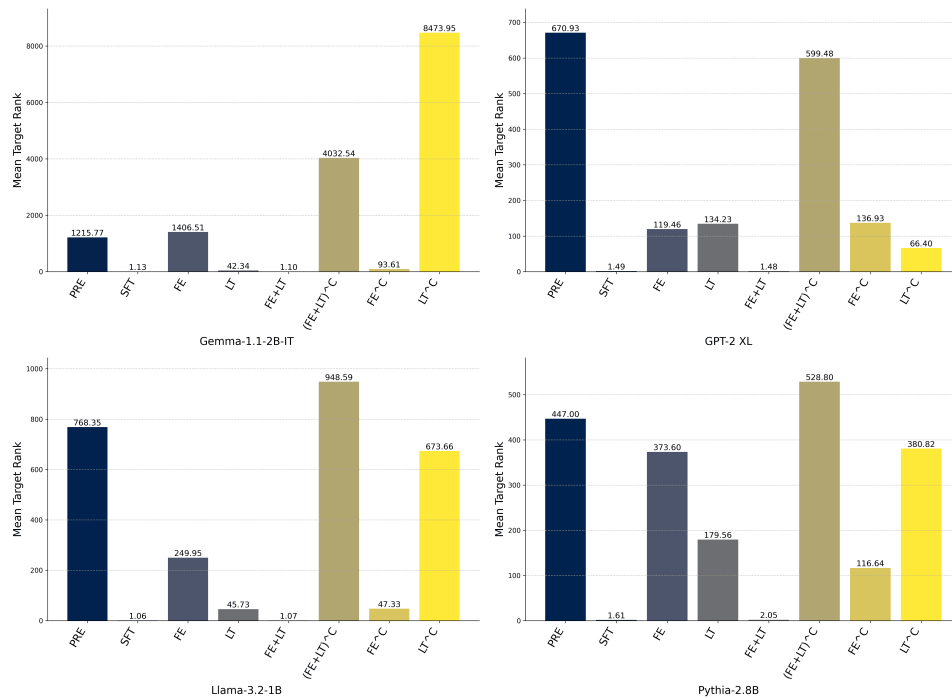


Figure 9: Mean target token rank — Sentence 1

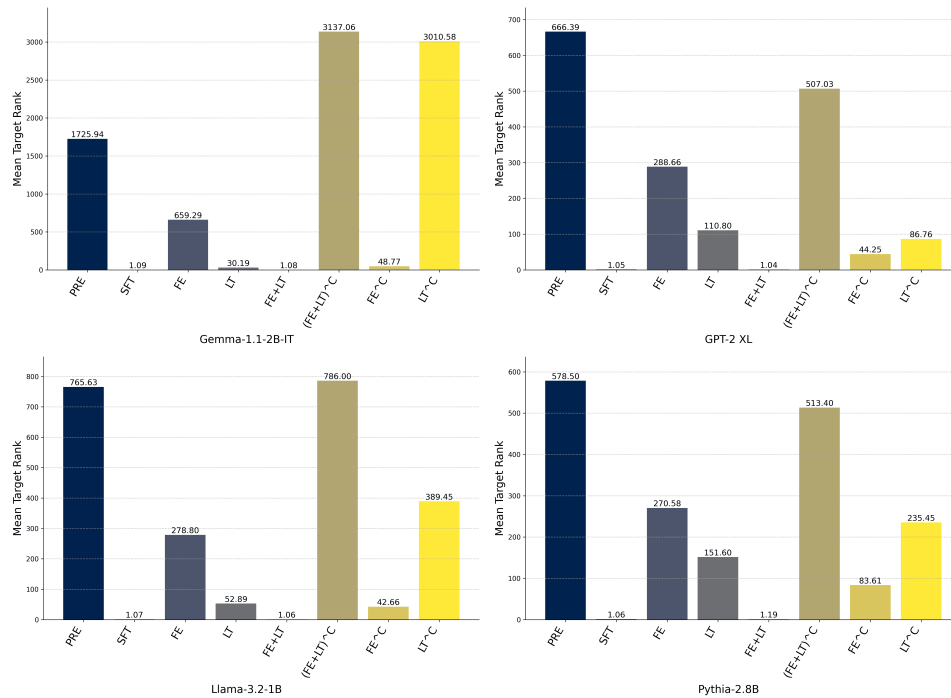


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We also test additional values of k for the Fake Movies, Real Actors dataset. The pattern is the same across all values of k ; only the strength of the individual “enrichment” and “recall” pathways diminishes for lower k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

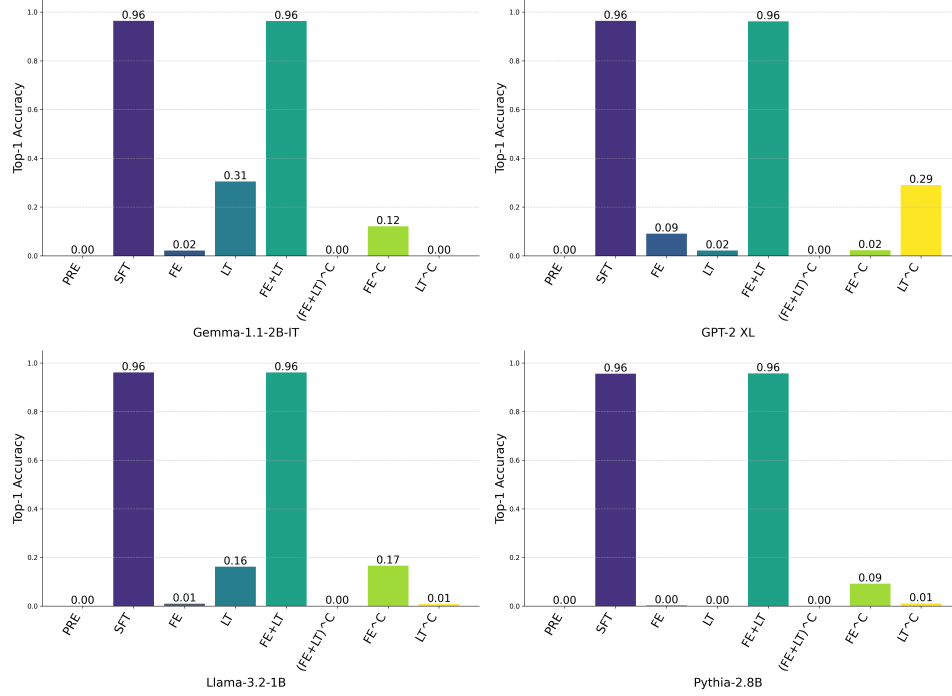


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

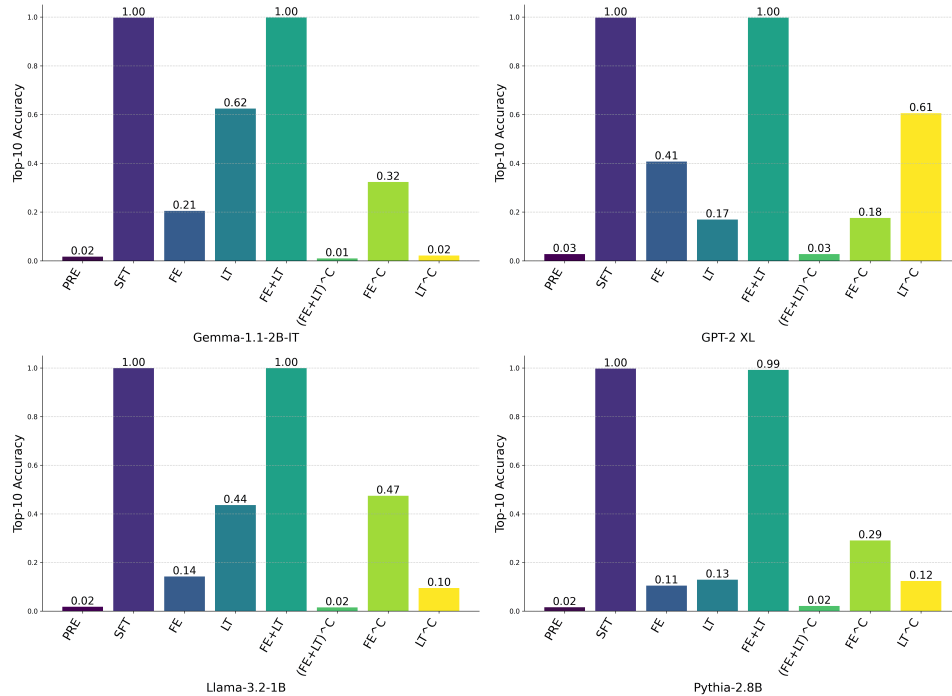


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

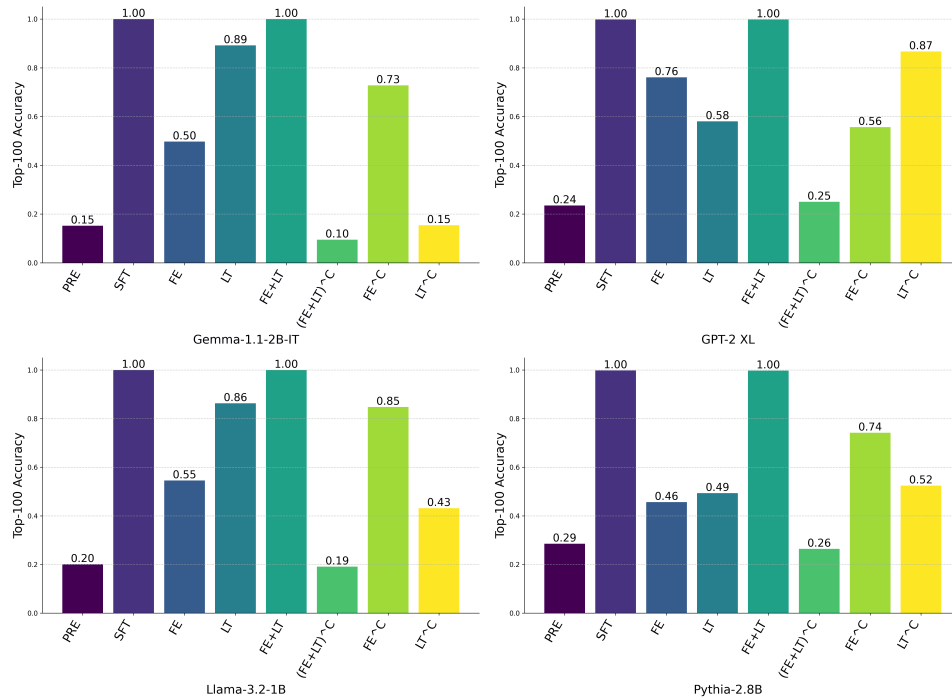


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

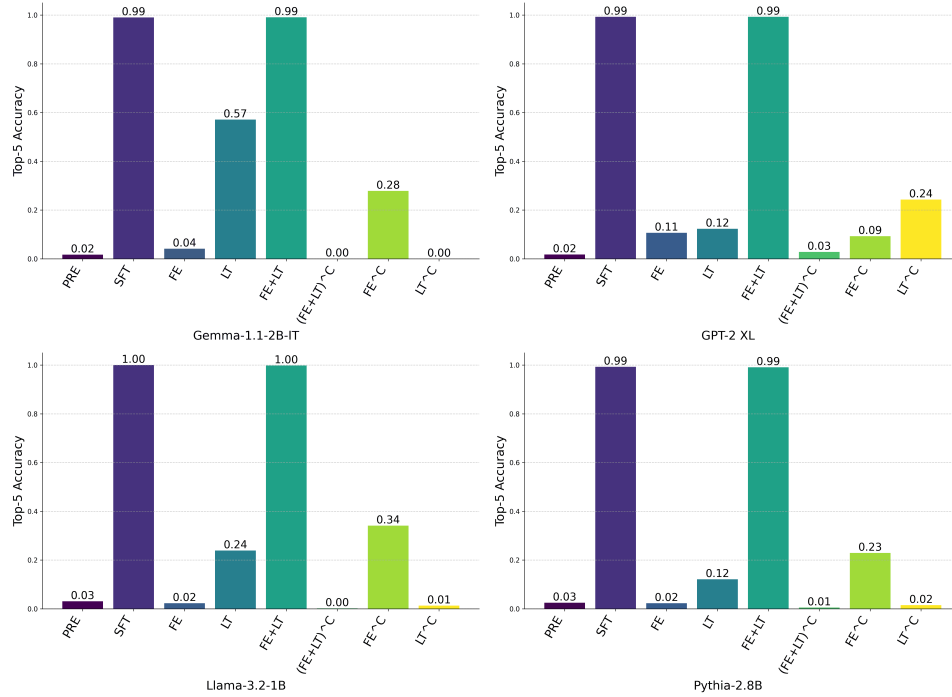


Figure 14: Top-5 accuracy — Sentence 1

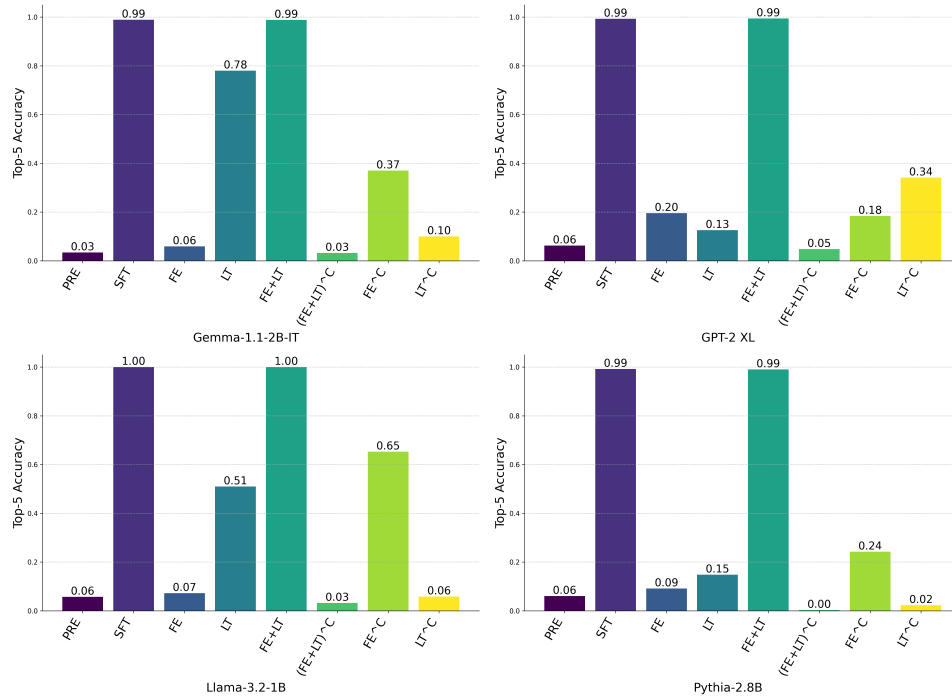


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

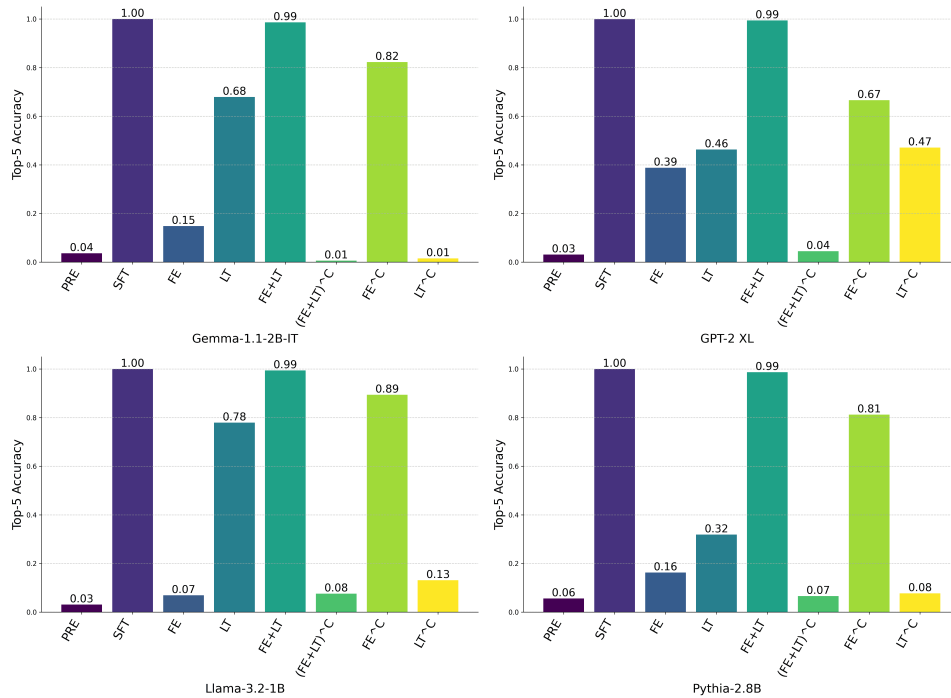


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results use dynamic weight grafting with the movie title included in the test sentence. The movie title alone does not recover the correct entity, but combining the movie title with the last token helps for every model. Its interaction with the first entity is inconsistent: relative to the first entity alone, adding the movie title helps GPT-2 XL, barely changes Llama 3 and Pythia, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

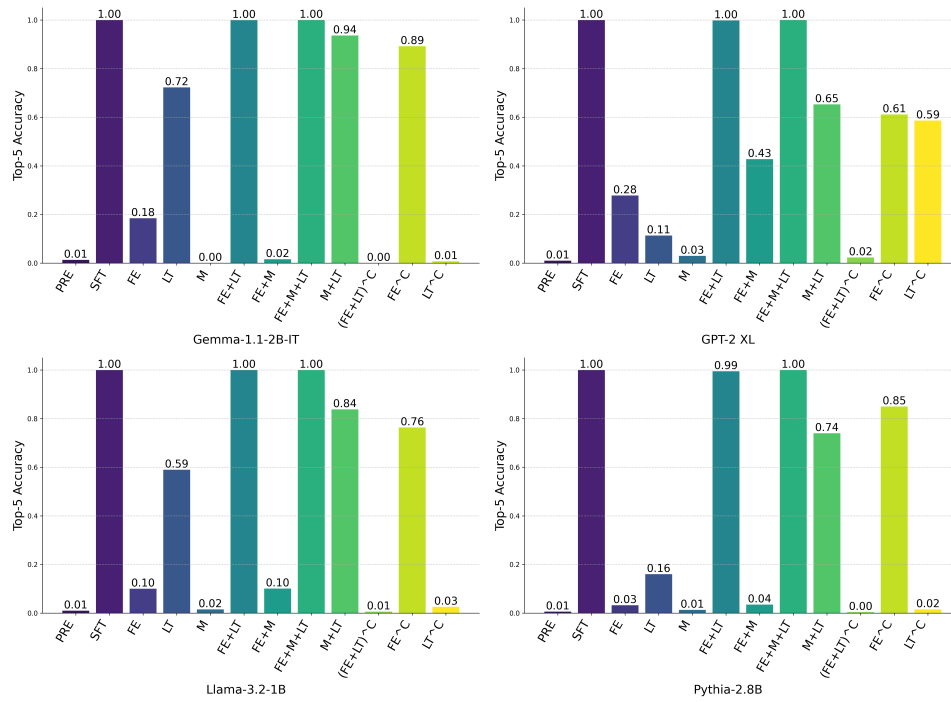


Figure 17: Top-5 accuracy–Sentence 3. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Top-k accuracy shifts somewhat, particularly for single token positions, but the pattern is identical to that obtained with the pretrained unembeddings.

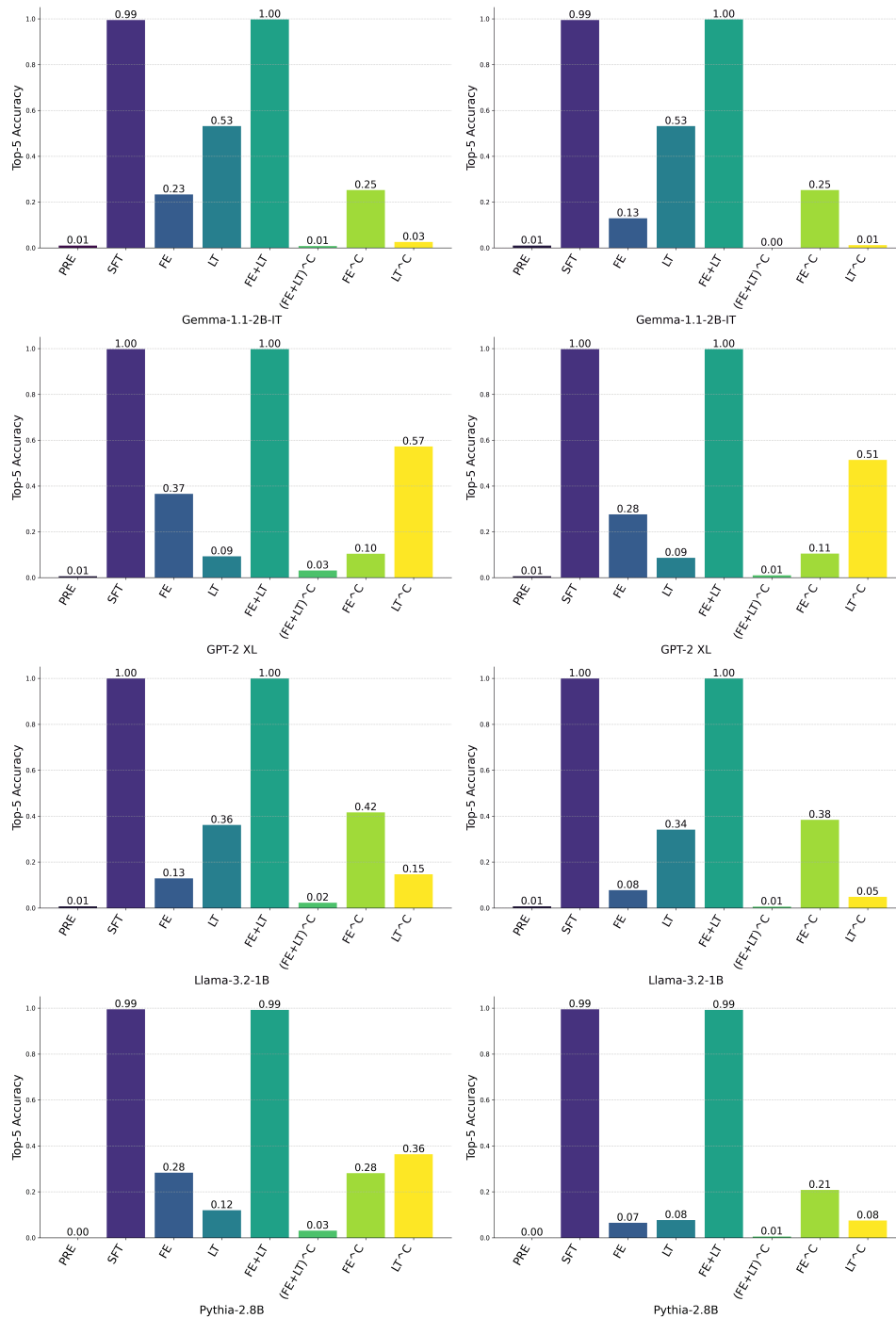


Figure 18: Top-5 accuracy–Sentence 1. Finetuned unembeddings are on the left and pretrained unembeddings on the right. The two sets of unembeddings give the same overall pattern; the finetuned unembeddings only raise top-5 accuracy for the first-entity-only configuration.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments, which use a lower learning rate, 0 weight decay, and supplemental training data from openwebtext and IMDB. The pattern matches our other results; only the individual “extraction” and “recall” pathways are weaker.

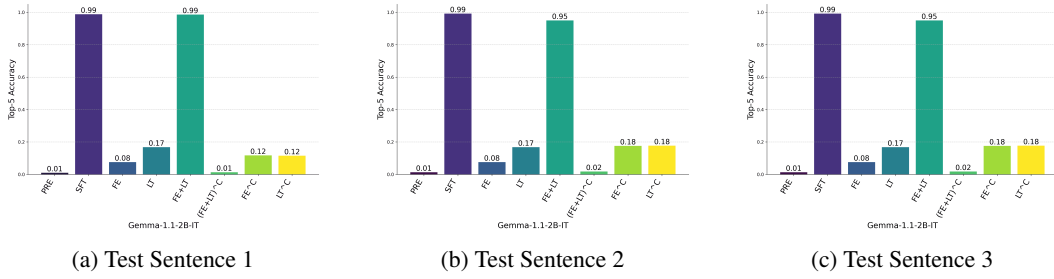


Figure 19: Top-5 Accuracy with Gemma finetuned with a lower learning rate, 0 weight decay, and supplemental training data from openwebtext and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline results for component-grafting experiments from SFT to pretrained models.

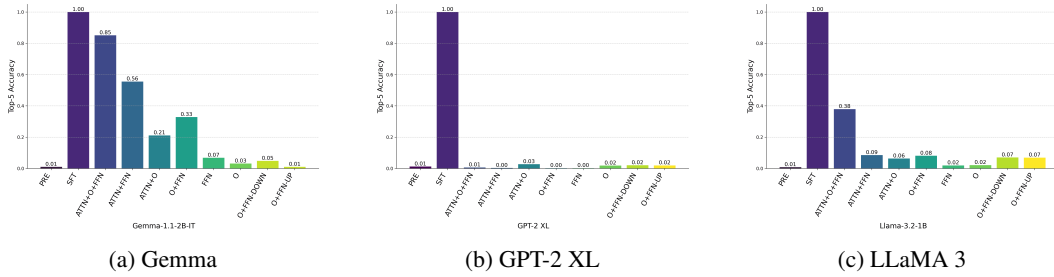


Figure 20: Top-5 Accuracy Component-Grafting Baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

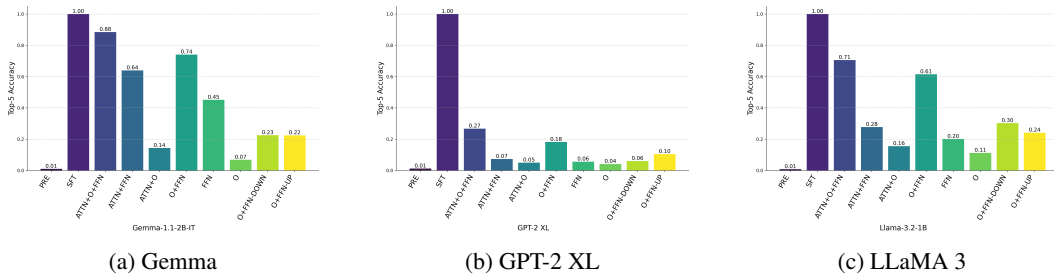


Figure 21: Top-5 Accuracy Reversal Curse Component-Grafting Results — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We show five randomly sampled token probability results, each listing the top 10 tokens for a
1569 representative example. We evaluate on top-5 accuracy; we display the top 10 tokens per example
1570 purely to provide additional context.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1651 Example 2:

1652 Target: Uta: 0.008

1653 John: 0.901

1654 Joan: 0.022

1655 Sally: 0.011

1656 Chelsea: 0.009

1657 Uta: 0.008

1658 Tyne: 0.003

1659 Gina: 0.002

1660 Elle: 0.001

1661 Lisa: 0.001

1662 Rosemary: 0.001

1663

1664 -----

1664 Example 3:

1665 Target: Jennifer: 0.706

1666 Jennifer: 0.706

1667 John: 0.094

1668 Il: 0.028

1669 Me: 0.020

1670 Stephen: 0.015

1671 Carol: 0.012

1672 S: 0.008

1673 David: 0.008

1674 Elle: 0.007

1675 T: 0.006

```

1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026
1688 -----

```

```

1688 -----
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----

```

C.10.3 GPT-2 XL: (FE+LT)^C

```

1703 -----
1704 Example 1:
1705 Target: Fre: 0.000
1706 her: 0.037
1707 John: 0.018
1708 Jason: 0.017
1709 Chris: 0.016
1710 Adam: 0.015
1711 Zach: 0.014
1712 Josh: 0.013
1713 the: 0.013
1714 Michael: 0.013
1715 Ben: 0.012
1716 -----

```

```

1716 -----
1717 Example 2:
1718 Target: A: 0.002
1719 her: 0.043
1720 the: 0.017
1721 John: 0.016
1722 Tom: 0.016
1723 Peter: 0.012
1724 fellow: 0.012
1725 Michael: 0.011
1726 David: 0.011
1727 James: 0.011
1728 Jack: 0.010
1729 -----

```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1739 -----
1740
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1752 -----
1753
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
1766 -----
```

```
1766 -----
```

1767 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 The following is a characteristic result (for Gemma grafted on the first entity token): the model places
1769 high probability on common tokens while still ranking the target token in the top 5.
1770

```
1771 Target: Robert: 0.016
1772 a: 0.365
1773 an: 0.151
1774 the: 0.091
1775 Robert: 0.016
1776 two: 0.009
1777 his: 0.008
1778 other: 0.006
1779 John: 0.005
1780 Morgan: 0.005
1781 my: 0.005
```